

Vulnerability Scan Report

NetShieldAI™

ZAP Scanner

Automated web application security assessment, including risk-ranked issues and remediation guidance.

TARGET URL:
http://testphp.vulnweb.com

SCAN DATE:
2026-01-02 19:00:34

REPORT GENERATED:
2026-01-02 19:00:34

Scan Overview

TOTAL ALERTS
17

HIGH RISK
3 HIGH

MEDIUM & LOW RISK
4 Medium / 4 Low

Prioritize remediation of **High** and **Medium** risk findings first, then address Low and Informational issues as part of hardening.

Risk Level	Count
High	3
Medium	4
Low	4
Informational	6
Total	17

Risk & Score Legend

Badge	Meaning
High Risk	Issues that can lead to compromise of data, authentication, or full system control.
Medium Risk	Exposures that may enable privilege escalation or sensitive information leakage.
Low Risk	Minor weaknesses or best-practice deviations with limited impact.
Informational	Non-exploit issues that still provide useful security context.
Predicted Score	Model-based severity estimate based on vulnerability features and context.

Detailed Findings

Each card represents one ZAP alert. Use the Description and Solution sections to understand the impact and remediation steps. References may include external standards or documentation.

SQL Injection - MySQL (Time Based)

High Risk

Predicted Score: 15.48

Confidence: Medium

URL: http://testphp.vulnweb.com/artists.php?artist=2

Description:

SQL injection may be possible.

Solution:

Do not trust client side input, even if there is client side validation in place.

In general, type check all data on the server side.

If the application uses JDBC, use PreparedStatement or CallableStatement, with parameters passed by '?'

If the application uses ASP, use ADO Command Objects with strong type checking and parameterized queries.

If database Stored Procedures can be used, use them.

Do not concatenate strings into queries in the stored procedure, or use 'exec', 'exec immediate', or equivalent functionality!

Do not create dynamic SQL queries using simple string concatenation.

Escape all data received from the client.

Apply an 'allow list' of allowed characters, or a 'deny list' of disallowed characters in user input.

Apply the principle of least privilege by using the least privileged database user possible.

In particular, avoid using the 'sa' or 'db-owner' database users. This does not eliminate SQL injection, but minimizes its impact.

Grant the minimum database access that is necessary for the application.

Reference:

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

XSLT Injection

Medium Risk

Predicted Score: 13.53

Confidence: Medium

URL: <http://testphp.vulnweb.com/showimage.php?file=%3Cxsl%3Avalue-of+select%3D%22document%28%27http%3A%2F%2Ftestphp.vulnweb.com%3A22%27%29%22%2F%3E>

Description:

Injection using XSL transformations may be possible, and may allow an attacker to read system information, read and write files, or execute arbitrary code.

Solution:

Sanitize and analyze every user input coming from any client-side.

Reference:

<https://book.hacktricks.wiki/en/pentesting-web/xslt-server-side-injection-extensible-stylesheet-language-transformations.html>

Predicted Score: 12.77

Confidence: Low

URL: http://testphp.vulnweb.com**Description:**

No Anti-CSRF tokens were found in a HTML submission form.

A cross-site request forgery is an attack that involves forcing a victim to send an HTTP request to a target destination without their knowledge or intent in order to perform an action as the victim. The underlying cause is application functionality using predictable URL/form actions in a repeatable way. The nature of the attack is that CSRF exploits the trust that a web site has for a user. By contrast, cross-site scripting (XSS) exploits the trust that a user has for a web site. Like XSS, CSRF attacks are not necessarily cross-site, but they can be. Cross-site request forgery is also known as CSRF, XSRF, one-click attack, session riding, confused deputy, and sea surf.

CSRF attacks are effective in a number of situations, including:

- * The victim has an active session on the target site.
- * The victim is authenticated via HTTP auth on the target site.
- * The victim is on the same local network as the target site.

CSRF has primarily been used to perform an action against a target site using the victim's privileges, but recent techniques have been discovered to disclose information by gaining access to the response. The risk of information disclosure is dramatically increased when the target site is vulnerable to XSS, because XSS can be used as a platform for CSRF, allowing the attack to operate within the bounds of the same-origin policy.

Solution:

Phase: Architecture and Design

Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid.

For example, use anti-CSRF packages such as the OWASP CSRFGuard.

Phase: Implementation

Ensure that your application is free of cross-site scripting issues, because most CSRF defenses can be bypassed using attacker-controlled script.

Phase: Architecture and Design

Generate a unique nonce for each form, place the nonce into the form, and verify the nonce upon receipt of the form. Be sure that the nonce is not predictable (CWE-330).

Note that this can be bypassed using XSS.

Identify especially dangerous operations. When the user performs a dangerous operation, send a separate confirmation request to ensure that the user intended to perform that operation.

Note that this can be bypassed using XSS.

Use the ESAPI Session Management control.

This control includes a component for CSRF.

Do not use the GET method for any request that triggers a state change.

Phase: Implementation

Check the HTTP Referer header to see if the request originated from an expected page. This could break legitimate functionality, because users or proxies may have disabled sending the Referer for privacy reasons.

Reference:

https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

<https://cwe.mitre.org/data/definitions/352.html>

Missing Anti-clickjacking Header

Medium Risk

Predicted Score: 11.0

Confidence: Medium

URL: <http://testphp.vulnweb.com>

Description:

The response does not protect against 'ClickJacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.

Solution:

Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. Ensure one of them is set on all web pages returned by your site/app.

If you expect the page to be framed only by pages on your server (e.g. it's part of a FRAMESET) then you'll want to use SAMEORIGIN, otherwise if you never expect the page to be framed, you should use DENY. Alternatively consider implementing Content Security Policy's "frame-ancestors" directive.

Reference:

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options>

Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)

Low Risk

Predicted Score: 9.88

Confidence: Medium

URL: http://testphp.vulnweb.com

Description:

The web/application server is leaking information via one or more "X-Powered-By" HTTP response headers. Access to such information may facilitate attackers identifying other frameworks/components your web application is reliant upon and the vulnerabilities such components may be subject to.

Solution:

Ensure that your web server, application server, load balancer, etc. is configured to suppress "X-Powered-By" headers.

Reference:

https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/01-Information_Gathering/08-Fingerprint_Web_Application_Framework
<https://www.troyhunt.com/shhh-dont-let-your-response-headers/>

Server Leaks Version Information via "Server" HTTP Response Header Field

Low Risk

Predicted Score: 9.88

Confidence: High

URL: <http://testphp.vulnweb.com>

Description:

The web/application server is leaking version information via the "Server" HTTP response header. Access to such information may facilitate attackers identifying other vulnerabilities your web/application server is subject to.

Solution:

Ensure that your web server, application server, load balancer, etc. is configured to suppress the "Server" header or provide generic details.

Reference:

<https://httpd.apache.org/docs/current/mod/core.html#servertokens>

[https://learn.microsoft.com/en-us/previous-versions/msp-n-p/ff648552\(v=pandp.10\)](https://learn.microsoft.com/en-us/previous-versions/msp-n-p/ff648552(v=pandp.10))

<https://www.troyhunt.com/shhh-dont-let-your-response-headers/>

Predicted Score: 9.53

Confidence: Medium

URL: <http://testphp.vulnweb.com/artists.php?artist=%3Cscript%3Ealert%281%29%3B%3C%2Fscript%3E>

Description:

Cross-site Scripting (XSS) is an attack technique that involves echoing attacker-supplied code into a user's browser instance. A browser instance can be a standard web browser client, or a browser object embedded in a software product such as the browser within WinAmp, an RSS reader, or an email client. The code itself is usually written in HTML/JavaScript, but may also extend to VBScript, ActiveX, Java, Flash, or any other browser-supported technology.

When an attacker gets a user's browser to execute his/her code, the code will run within the security context (or zone) of the hosting web site. With this level of privilege, the code has the ability to read, modify and transmit any sensitive data accessible by the browser. A Cross-site Scripted user could have his/her account hijacked (cookie theft), their browser redirected to another location, or possibly shown fraudulent content delivered by the web site they are visiting. Cross-site Scripting attacks essentially compromise the trust relationship between a user and the web site. Applications utilizing browser object instances which load content from the file system may execute code under the local machine zone allowing for system compromise.

There are three types of Cross-site Scripting attacks: non-persistent, persistent and DOM-based.

Non-persistent attacks and DOM-based attacks require a user to either visit a specially crafted link laced with malicious code, or visit a malicious web page containing a web form, which when posted to the vulnerable site, will mount the attack. Using a malicious form will oftentimes take place when the vulnerable resource only accepts HTTP POST requests. In such a case, the form can be submitted automatically, without the victim's knowledge (e.g. by using JavaScript). Upon clicking on the malicious link or submitting the malicious form, the XSS payload will get echoed back and will get interpreted by the user's browser and execute. Another technique to send almost arbitrary requests (GET and POST) is by using an embedded client, such as Adobe Flash.

Persistent attacks occur when the malicious code is submitted to a web site where it's stored for a period of time. Examples of an attacker's favorite targets often include message board posts, web mail messages, and web chat software. The unsuspecting user is not required to interact with any additional site/link (e.g. an attacker site or a malicious link sent via email), just simply view the web page containing the code.

Solution:

Phase: Architecture and Design

Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid.

Examples of libraries and frameworks that make it easier to generate properly encoded output include Microsoft's Anti-XSS library, the OWASP ESAPI Encoding module, and Apache Wicket.

Phases: Implementation; Architecture and Design

Understand the context in which your data will be used and the encoding that will be expected. This is especially important when transmitting data between different components, or when generating outputs that can contain multiple encodings at the same time, such as web pages or multi-part mail messages.

Study all expected communication protocols and data representations to determine the required encoding strategies.

For any data that will be output to another web page, especially any data that was received from external inputs, use the appropriate encoding on all non-alphanumeric characters.

Consult the XSS Prevention Cheat Sheet for more details on the types of encoding and escaping that are needed.

Phase: Architecture and Design

For any security checks that are performed on the client side, ensure that these checks are duplicated on the server side, in order to avoid CWE-602. Attackers can bypass the client-side checks by modifying values after the checks have been performed, or by changing the client to remove the client-side checks entirely. Then, these modified values would be submitted to the server.

If available, use structured mechanisms that automatically enforce the separation between data and code. These mechanisms may be able to provide the relevant quoting, encoding, and validation automatically, instead of relying on the developer to provide this capability at every point where output is generated.

Phase: Implementation

For every web page that is generated, use and specify a character encoding such as ISO-8859-1 or UTF-8. When an encoding is not specified, the web browser may choose a different encoding by guessing which encoding is actually being used by the web page. This can cause the web browser to treat certain sequences as special, opening up the client to subtle XSS attacks. See CWE-116 for more mitigations related to encoding/escaping.

To help mitigate XSS attacks against the user's session cookie, set the session cookie to be HttpOnly. In browsers that support the HttpOnly feature (such as more recent versions of Internet Explorer and Firefox), this attribute can prevent the user's session cookie from being accessible to malicious client-side scripts that use document.cookie. This is not a complete solution, since HttpOnly is not supported by all browsers. More importantly, XMLHttpRequest and other powerful browser technologies provide read access to HTTP headers, including the Set-Cookie header in which the HttpOnly flag is set.

Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use an allow list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. Do not rely exclusively on looking for malicious or malformed inputs (i.e., do not rely on a deny list). However, deny lists can be useful for detecting potential attacks or determining which inputs are so malformed that they should be rejected outright.

When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if you are expecting colors such as "red" or "blue."

Ensure that you perform input validation at well-defined interfaces within the application. This will help protect the application even if a component is reused or moved elsewhere.

Reference:

<https://owasp.org/www-community/attacks/xss/>

<https://cwe.mitre.org/data/definitions/79.html>

User Controllable HTML Element Attribute (Potential XSS)

[Info Risk](#)

Predicted Score: 9.39

Confidence: Low

URL: http://testphp.vulnweb.com/guestbook.php

Description:

This check looks at user-supplied input in query string parameters and POST data to identify where certain HTML attribute values might be controlled. This provides hot-spot detection for XSS (cross-site scripting) that will require further review by a security analyst to determine exploitability.

Solution:

Validate all input and sanitize output it before writing to any HTML attributes.

Reference:

https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html

GET for POST

[Info Risk](#)

Predicted Score: 6.45

Confidence: High

URL: http://testphp.vulnweb.com/cart.php

Description:

A request that was originally observed as a POST was also accepted as a GET. This issue does not represent a security weakness unto itself, however, it may facilitate simplification of other attacks. For example if the original POST is subject to Cross-Site Scripting (XSS), then this finding may indicate that a simplified (GET based) XSS may also be possible.

Solution:

Ensure that only POST is accepted where POST is expected.

Content Security Policy (CSP) Header Not Set

Medium Risk

Predicted Score: 5.25

Confidence: High

URL: http://testphp.vulnweb.com

Description:

Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.

Solution:

Ensure that your web server, application server, load balancer, etc. is configured to set the Content-Security-Policy header.

Reference:

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP>

https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html

<https://www.w3.org/TR/CSP/>

<https://w3c.github.io/webappsec-csp/>

<https://web.dev/articles/csp>

<https://caniuse.com/#feat=contentsecuritypolicy>

<https://content-security-policy.com/>

X-Content-Type-Options Header Missing

Low Risk

Predicted Score: 5.25

Confidence: Medium

URL: http://testphp.vulnweb.com

Description:

The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.

Solution:

Ensure that the application/web server sets the Content-Type header appropriately, and that it sets the X-Content-Type-Options header to 'nosniff' for all web pages.

If possible, ensure that the end user uses a standards-compliant and modern web browser that does not perform MIME-sniffing at all, or that can be directed by the web application/web server to not perform MIME-sniffing.

Reference:

[https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/gg622941\(v=vs.85\)](https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/gg622941(v=vs.85))

<https://owasp.org/www-community/Security-Headers>

Predicted Score: N/A

Confidence: Medium

URL: http://testphp.vulnweb.com/artists.php?artist=%27**Description:**

SQL injection may be possible.

Solution:

Do not trust client side input, even if there is client side validation in place.

In general, type check all data on the server side.

If the application uses JDBC, use PreparedStatement or CallableStatement, with parameters passed by '?'

If the application uses ASP, use ADO Command Objects with strong type checking and parameterized queries.

If database Stored Procedures can be used, use them.

Do **not** concatenate strings into queries in the stored procedure, or use 'exec', 'exec immediate', or equivalent functionality!

Do not create dynamic SQL queries using simple string concatenation.

Escape all data received from the client.

Apply an 'allow list' of allowed characters, or a 'deny list' of disallowed characters in user input.

Apply the principle of least privilege by using the least privileged database user possible.

In particular, avoid using the 'sa' or 'db-owner' database users. This does not eliminate SQL injection, but minimizes its impact.

Grant the minimum database access that is necessary for the application.

Reference:

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

ZAP is Out of Date

Low Risk

Predicted Score: N/A

Confidence: High

URL: <http://testphp.vulnweb.com/userinfo.php>

Description:

The version of ZAP you are using to test your app is out of date and is no longer being updated.
The risk level is set based on how out of date your ZAP version is.

Solution:

Download the latest version of ZAP from <https://www.zaproxy.org/download/> and install it.

Reference:

<https://www.zaproxy.org/download/>

Authentication Request Identified

Info Risk

Predicted Score: N/A

Confidence: Low

URL: <http://testphp.vulnweb.com/secured/newuser.php>

Description:

The given request has been identified as an authentication request. The 'Other Info' field contains a set of key=value lines which identify any relevant fields. If the request is in a context which has an Authentication Method set to "Auto-Detect" then this rule will change the authentication to match the request identified.

Solution:

This is an informational alert rather than a vulnerability and so there is nothing to fix.

Reference:

<https://www.zaproxy.org/docs/desktop/addons/authentication-helper/auth-req-id/>

Charset Mismatch (Header Versus Meta Content-Type Charset)

Info Risk

Predicted Score: N/A

Confidence: Low

URL: http://testphp.vulnweb.com

Description:

This check identifies responses where the HTTP Content-Type header declares a charset different from the charset defined by the body of the HTML or XML. When there's a charset mismatch between the HTTP header and content body Web browsers can be forced into an undesirable content-sniffing mode to determine the content's correct character set.

An attacker could manipulate content on the page to be interpreted in an encoding of their choice. For example, if an attacker can control content at the beginning of the page, they could inject script using UTF-7 encoded text and manipulate some browsers into interpreting that text.

Solution:

Force UTF-8 for all text content in both the HTTP header and meta tags in HTML or encoding declarations in XML.

Reference:

https://code.google.com/p/browsersec/wiki/Part2#Character_set_handling_and_detection

Modern Web Application

Info Risk

Predicted Score: N/A

Confidence: Medium

URL: http://testphp.vulnweb.com/AJAX/index.php

Description:

The application appears to be a modern web application. If you need to explore it automatically then the Ajax Spider may well be more effective than the standard one.

Solution:

This is an informational alert and so no changes are required.

User Agent Fuzzer

[Info Risk](#)

Predicted Score: N/A

Confidence: Medium

URL: <http://testphp.vulnweb.com/AJAX>

Description:

Check for differences in response based on fuzzed User Agent (eg. mobile sites, access as a Search Engine Crawler). Compares the response statuscode and the hashcode of the response body with the original response.

Solution:

Reference:

<https://owasp.org/wstg>

Generated by NetShieldAI Reporting Engine